Karadeniz Teknik Üniversitesi
Fen Fakültesi – Bilgisayar Bilimleri Bölümü

Hızlı Başvuru Kağıdı #6: Carpishma Algilama ve Fizik Temelleri

PyGame ile Oyun Geliştirme – Hafta 6

Rect Sinifi

Rect Olusturma

Yontem	Aciklama
`Rect(x,y,w,h)`	Dogrudan olustur
`Surface.get_rect()`	Surface'ten al
`get_rect(center=..)`	Konumlu olustur

Rect Ozellikleri

Ozellik	Aciklama
`x, y`	Sol ust kose
`width, height`	Boyutlar
`center, centerx, centery`	Merkez noktasi
`top, bottom, left, right`	Kenar degerleri
`topleft, topright`	Kose ciftleri
`midtop, midbottom`	Kenar orta noktalar
`size`	(width, height)

Rect Kullanimi

```python
# Dogrudan olusturma
kutu = pygame.Rect(100, 200, 50, 50)

# Surface'ten alma
gorsel = pygame.image.load(
    "gemi.png").convert_alpha()
rect = gorsel.get_rect(center=(400,300))

# Ozelliklere erisim
print(rect.center)   # (400, 300)
print(rect.topleft)  # (375, 275)
print(rect.size)     # (50, 50)
```

Rect Metodlari

Metot	Aciklama
`move(x, y)`	Yeni Rect dondurur
`move_ip(x, y)`	Yerinde kaydirma
`inflate(w, h)`	Boyutu degistir (yeni)
`inflate_ip(w, h)`	Yerinde boyutlandir
`clamp(rect)`	Sinir icerisine al
`clamp_ip(rect)`	Yerinde sinirla
`union(rect)`	Birlestir
`clip(rect)`	Kesisim Rect'i

Rect Carpismasi

– Rect vs Rect

Iki dikdortgenin cakisip cakismadigini kontrol eder.

Donus degeri: `True` (cakisiyor) / `False`

`colliderect()` Kullanimi

```python
oyuncu = pygame.Rect(100, 100, 50, 50)
dusman = pygame.Rect(130, 120, 40, 40)

if oyuncu.colliderect(dusman):
    print("Carpishma var!")
    can -= 1
```

– Rect vs Nokta

Bir noktanin (x, y) dikdortgen icerisinde olup olmadigini kontrol eder. Fare tiklamalari icin idealdir.

`collidepoint()` ile Fare Tiklama

```python
buton = pygame.Rect(300, 200, 150, 50)

for olay in pygame.event.get():
    if olay.type == pygame.MOUSEBUTTONDOWN:
        if buton.collidepoint(olay.pos):
            print("Butona tiklandi!")
```

Diger Carpishma Metodlari

Metot	Aciklama
`collidelist(list)`	Listedeki ilk carpisan indeks
`collidelistall(list)`	Tum carpisan indeksler
`contains(rect)`	Tamamen iceriyor mu?

Sprite Carpishma Fonksiyonlari

– Sprite vs Grup

Bir sprite'in bir gruptaki hangi sprite'larla carpistigini bulur.

```python
pygame.sprite.spritecollide(sprite, group, dokill)
```

`dokill=True` → carpisan sprite'lar gruptan silinir

`dokill=False` → sadece liste doner

`spritecollide()` Kullanimi

```python
# Oyuncu ile carpisan dusmanlar
vurulanlar = pygame.sprite.spritecollide(
    oyuncu, dusman_grup, True)

for dusman in vurulanlar:
    puan += 10
    patlama_efekti(dusman.rect.center)
```

– Grup vs Grup

Iki gruptaki sprite'larin birbirleriyle carpismasini kontrol eder.

```python
pygame.sprite.groupcollide(
group1, group2, dokill1, dokill2)
```

Donus: `{sprite1: [sprite2, ...], ...}` sozlugu

`groupcollide()` Kullanimi

```python
# Mermiler vs dusmanlar
isaretler = pygame.sprite.groupcollide(
    mermi_grup, dusman_grup, True, True)

for mermi, dusmanlar in isaretler.items():
    for d in dusmanlar:
        puan += 50
```

Carpishma Yontemleri

Yontem	Aciklama
Rect (varsayilan)	Dikdortgen kutu, en hizli
`collide_circle`	Daire tabanli, orta hiz
`collide_mask`	Piksel tabanli, en dogru

`collide_mask` Kullanimi

```python
# Sprite'a mask ekle
class Oyuncu(pygame.sprite.Sprite):
    def __init__(self):
        super().__init__()
        self.image = pygame.image.load(
            "gemi.png").convert_alpha()
        self.rect = self.image.get_rect()
        self.mask = pygame.mask.from_surface(
            self.image)

# Mask ile carpishma kontrolu
vurulanlar = pygame.sprite.spritecollide(
    oyuncu, dusmanlar, False,
    pygame.sprite.collide_mask)
```

Mask Performans Uyarisi

`collide_mask` en dogru sonucu verir ama en yavas yontemdir.

Strateji: Once `colliderect()` ile on eleme yap, sonra `collide_mask` ile hassas kontrol uygula.

collide_circle Kullanimi

Daire carpismasi icin sprite'a `radius` ozelligi ekle:
`self.radius = 20`
Eklenmezse otomatik hesaplanir (`image` boyutundan).

Temel Fizik Simulasyonu

Yercekimi

Yercekimi = Her karede hiza sabit deger ekleme

`hiz_y += YERCEKIMI`

`rect.y += hiz_y`

Tipik `YERCEKIMI` degeri: `0.3 – 1.0`

Yercekimi Uygulamasi

```
YERCEKIMI = 0.5
ZEMIN_Y = 550

class Top(pygame.sprite.Sprite):
    def __init__(self, x, y):
        super().__init__()
        self.image = pygame.Surface((30, 30),
            pygame.SRCALPHA)
        pygame.draw.circle(self.image,
            (255,100,0), (15,15), 15)
        self.rect = self.image.get_rect(
            center=(x, y))
        self.hiz_y = 0

    def update(self):
        self.hiz_y += YERCEKIMI
        self.rect.y += self.hiz_y
        if self.rect.bottom >= ZEMIN_Y:
            self.rect.bottom = ZEMIN_Y
            self.hiz_y = 0
```

Momentum ve Surtunme

Kavram	Formul
Hiz	`konum += hiz`
Ivme	`hiz += ivme`
Yercekimi	`hiz_y += g`
Surtunme	`hiz_x *= 0.95`
Geri Sekme	`hiz_y *= -sekme`

Surtunme ve Geri Sekme

```
SURTUNME = 0.95
SEKME_KATSAYISI = 0.7

def update(self):
    # Yercekimi
    self.hiz_y += YERCEKIMI
    # Surtunme (yatay)
    self.hiz_x *= SURTUNME
    # Konum guncelle
    self.rect.x += self.hiz_x
    self.rect.y += self.hiz_y
    # Zemin carpismasi + geri sekme
    if self.rect.bottom >= ZEMIN_Y:
        self.rect.bottom = ZEMIN_Y
        self.hiz_y *= -SEKME_KATSAYISI
        # Kucuk sektirmeleri durdur
        if abs(self.hiz_y) < 1:
            self.hiz_y = 0
```

Hasar Sistemi ve Yok Etme

Hasar Sistemi

```
class Dusman(pygame.sprite.Sprite):
    def __init__(self, x, y, can=3):
        super().__init__()
        self.image = pygame.Surface((40,40))
        self.image.fill((200, 30, 30))
        self.rect = self.image.get_rect(
            center=(x, y))
        self.can = can

    def hasar_al(self, miktar=1):
        self.can -= miktar
        if self.can <= 0:
            self.kill()  # Tum gruplardan sil
            return True  # Yok edildi
        return False
```

Breakout Mantigi

Eleman	Davranis
Raket	Fare/klavye ile yatay hareket
Top	`hiz_x`, `hiz_y` ile hareket
Tuglalar	Carpismada hasar al / yok ol
Duvarlar	`hiz_x *= -1` (yan), top reset
Tavan	`hiz_y *= -1`

Top Geri Sekme (Breakout)

```
def update(self):
    self.rect.x += self.hiz_x
    self.rect.y += self.hiz_y

    # Yan duvarlar
    if self.rect.left <= 0 or \
       self.rect.right >= GENISLIK:
        self.hiz_x *= -1
    # Tavan
    if self.rect.top <= 0:
        self.hiz_y *= -1

    # Raket carpismasi
    if self.rect.colliderect(raket.rect):
        self.hiz_y *= -1
        self.rect.bottom = raket.rect.top
```

Sik Yapilan Hatalar

Hatalar ve Cozumler

Hata	Cozum
`mask` tanimlamama	`collide_mask` icin zorunlu
Tunelden gecme	Kucuk adimlarla kontrol et
Sonsuz sekme	`abs(hiz) < 1` ise durdur
`dokill` yanlis	`True`: siler, `False`: silmez
Carpismada takilma	Konum duzelt, sonra yonu cevir
Fizik dt'siz	`dt = clock.tick(60)/1000`

Fizik Motoru Sablonu

Fizik dongusu sirasi:
1. Kuvvetleri uygula (yercekimi, surtunme)
2. Hizi guncelle (`hiz += ivme`)
3. Konumu guncelle (`konum += hiz`)
4. Carpishma kontrolu yap
5. Carpishma tepkisi uygula (sekme, hasar)